

Set

List vs Set

항목	List (리스트)	Set (셋)
한 줄 설명	순서가 있는 목록	중복 없는 모음
중복 값	허용	불허
순서	있음 (입력 순서 유지)	없음
비유	줄 서있는 대기열	출석부 (이름 중복 없음)
언제 쓰나?	순서가 중요할 때	중복을 없애고 싶을 때

1. Redis Set

Set은 순서 없이 중복되지 않는 문자열을 저장합니다. 특정 값의 포함 여부를 빠르게 확인하거나 여러 집합의 관계를 계산할 때 사용합니다.

```
DEL demo:set          # demo:set 키 삭제 (초기화)
SADD demo:set apple banana apple # Set에 apple, banana, apple 추가 (중복 apple은 무시됨)
SMEMBERS demo:set      # Set의 모든 멤버 조회
SCARD demo:set         # Set의 멤버 수 반환 (2)
SISMEMBER demo:set apple # apple이 Set에 있는지 확인 (1=있음, 0=없음)
```

`SADD`의 결과는 실제로 새로 추가된 값의 개수입니다. 위 예에서 `apple`은 두 번 전달되지만 한 번만 저장됩니다.



[Databases](#) > My Redis Stack Database db0  

0.29 % | 0 | 1 MB | 0 | 2  Insights

1 DEL demo:set

2 SADD demo:set apple banana apple

3 SMEMBERS demo:set

4 SCARD demo:set

5 SISMEMBER demo:set apple

-r

Clear Results

SISMEMBER demo:set apple	Jun 23, 10:31:32	5.909 msec				
(integer) 1						
SCARD demo:set	Jun 23, 10:31:32	6.128 msec				
(integer) 2						
SMEMBERS demo:set	Jun 23, 10:31:32	6.69 msec				
1) "apple"						
2) "banana"						
SADD demo:set apple banana apple	Jun 23, 10:31:32	7.094 msec				
(integer) 2						
DEL demo:set	Jun 23, 10:31:32	3.436 msec				
(integer) 0						

2. 추가, 삭제, 무작위 선택

```
SADD event:participants user1 user2 user3 # Set에 user1, user2, user3 추가
SREM event:participants user2             # Set에서 user2 제거
SISMEMBER event:participants user2        # user2가 Set에 있는지 확인 (0=없음)
SRANDMEMBER event:participants            # 랜덤으로 멤버 1개 조회 (제거 안 함)
SPOP event:participants                   # 랜덤으로 멤버 1개 꺼내기 (제거함)
```

- SRANDMEMBER : 무작위 값을 반환하지만 삭제하지 않음
- SPOP : 무작위 값을 반환하고 집합에서 삭제

경품 추천에서 후보를 유지하려면 SRANDMEMBER , 한 번 뽑힌 사람을 제외하려면 SPOP 을 사용할 수 있습니다.



[Databases](#) > My Redis Stack Database db0  

0.26 % | 0 | 1 MB | 1 | 3  Insights

 1 SADD event:participants user1 user2 user3
 2 SREM event:participants user2
 3 SISEMEMBER event:participants user2
 4 SRANDMEMBER event:participants
 5 SPOP event:participants

-r





Clear Results

SPOP event:participants	Jun 23, 10:33:19	2.088 msec				
"user1"						
SRANDMEMBER event:participants	Jun 23, 10:33:19	0.627 msec				
"user3"						
SISEMEMBER event:participants user2	Jun 23, 10:33:19	0.752 msec				
(integer) 0						
SREM event:participants user2	Jun 23, 10:33:19	0.91 msec				
(integer) 1						
SADD event:participants user1 user2 user3	Jun 23, 10:33:19	1.222 msec				
(integer) 3						

3. 집합 연산

```
SADD user:1:interests redis python database
SADD user:2:interests redis java database
SINTER user:1:interests user:2:interests
SUNION user:1:interests user:2:interests
SDIFF user:1:interests user:2:interests
```

user1의 관심사 추가: redis, python, database
user2의 관심사 추가: redis, java, database
교집합: 둘 다 가진 것 → {redis, database}
합집합: 둘 중 하나라도 가진 것 → {redis, python, database, java}
차집합: user1만 가진 것 → {python}

명령어	의미	결과
SINTER	교집합	redis, database
SUNION	합집합	redis, python, database, java
SDIFF	차집합 (기준 - 나머지)	python

결과를 새 Set에 저장하려면 `SINTERSTORE` , `SUNIONSTORE` , `SDIFFSTORE` 를 사용합니다.



[Databases](#) > My Redis Stack Database db0  

0.29 % | 0 | 2 MB | 2 | 3  Insights



1 SADD user:1:interests redis python database

2 SADD user:2:interests redis java database

3 SINTER user:1:interests user:2:interests

4 SUNION user:1:interests user:2:interests

5 SDIFF user:1:interests user:2:interests

-r





SDIFF user:1:interests user:2:interests	Jun 23, 10:35:28	2.576 msec	   
1) "python"			
SUNION user:1:interests user:2:interests	Jun 23, 10:35:28	2.828 msec	   
1) "redis"			
2) "python"			
3) "database"			
4) "java"			
SINTER user:1:interests user:2:interests	Jun 23, 10:35:28	3.155 msec	   
1) "redis"			
2) "database"			
SADD user:2:interests redis java database	Jun 23, 10:35:28	3.355 msec	   
(integer) 3			
SADD user:1:interests redis python database	Jun 23, 10:35:28	3.767 msec	   
(integer) 3			

4. 게시물 좋아요

게시글마다 좋아요를 누른 사용자 번호를 Set으로 저장합니다.

```
SADD article:100:likes 7      # 게시물 100에 user7 좋아요 추가
SADD article:100:likes 8      # 게시물 100에 user8 좋아요 추가
SADD article:100:likes 7      # user7 좋아요 중복 추가 (무시됨)
SCARD article:100:likes       # 좋아요 수 조회 → 2 (중복 제거된 결과)
SISMEMBER article:100:likes 7 # user7이 좋아요 눌렀는지 확인 (1=눌렀음)
SREM article:100:likes 7      # user7 좋아요 취소
```

사용자 7이 여러 번 요청해도 한 번만 저장됩니다. `SCARD` 는 좋아요 수,
`SISMEMBER` 는 현재 사용자의 좋아요 여부를 알려 줍니다.



Databases > My Redis Stack Database db0  



0.25 % | 0 | 2 MB | 4 | 3

Insights

```
1 SADD article:100:likes 7
2 SADD article:100:likes 8
3 SADD article:100:likes 7
4 SCARD article:100:likes
5 SISMEMBER article:100:likes 7
6 SREM article:100:likes 7
```

SREM article:100:likes 7	Jun 23, 10:38:21	1.959 msec				
(integer) 1						
SISMEMBER article:100:likes 7	Jun 23, 10:38:21	6.992 msec				
(integer) 1						
SCARD article:100:likes	Jun 23, 10:38:21	7.06 msec				
(integer) 2						
SADD article:100:likes 7	Jun 23, 10:38:21	7.152 msec				
(integer) 0						
SADD article:100:likes 8	Jun 23, 10:38:21	7.356 msec				
(integer) 1						
SADD article:100:likes 7	Jun 23, 10:38:21	4.475 msec				
(integer) 1						

5. Set을 선택하는 기준

Set이 적합한 질문은 다음과 같습니다.

- 이 사용자가 이미 참여했는가?
- 고유 방문자는 누구인가?
- 두 사용자의 공통 관심사는 무엇인가?
- 중복 없이 태그를 관리하려면 어떻게 할까?

순위나 점수가 필요하면 `Sorted Set`, 필드와 값이 필요하면 `Hash` 를 사용합니다.